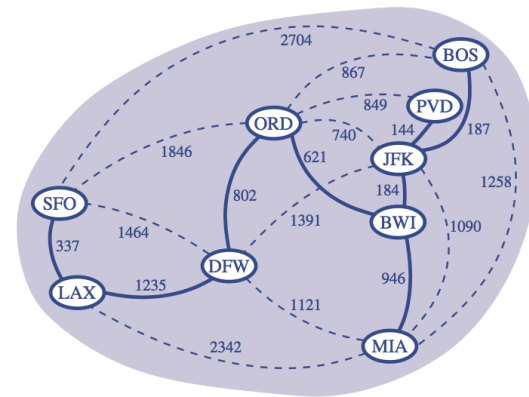


Minimum Spanning Trees



Minimum Spanning Trees

Spanning subgraph

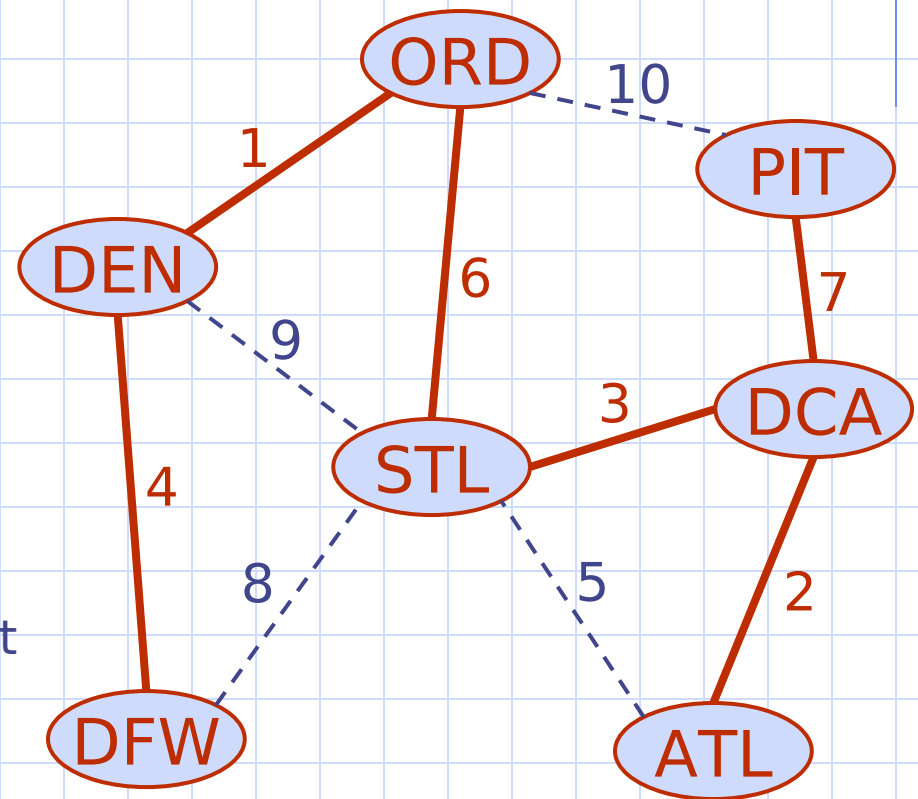
- Subgraph of a graph G containing all the vertices of G

Spanning tree

- Spanning subgraph that is itself a (free) tree

Minimum spanning tree (MST)

- Spanning tree of a weighted graph with minimum total edge weight
- Applications
 - Communications networks
 - Transportation networks



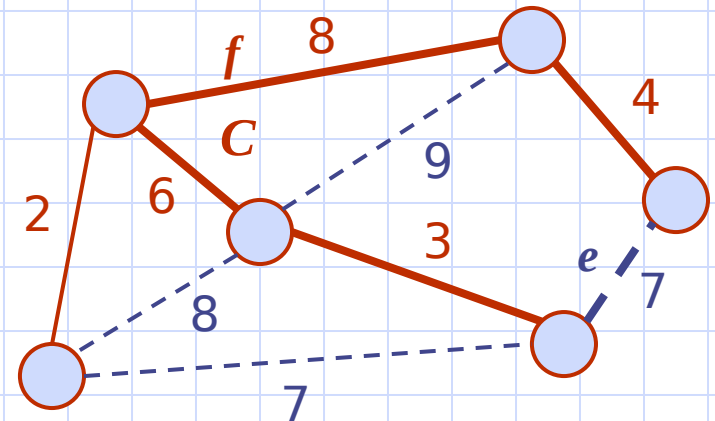
Cycle Property

Cycle Property:

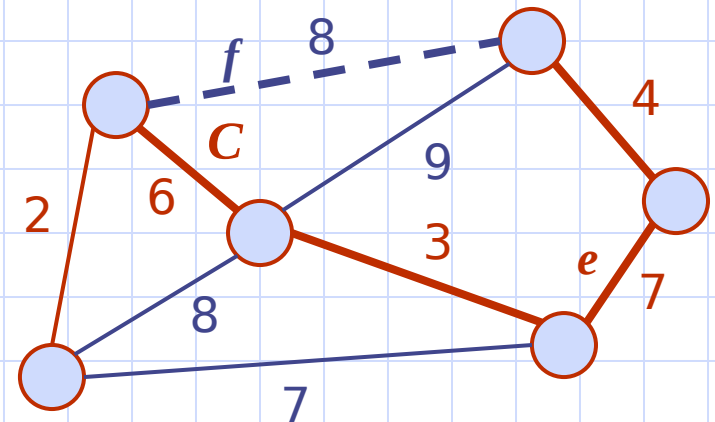
- Let T be a minimum spanning tree of a weighted graph G
- Let e be an edge of G that is not in T and C let be the cycle formed by e with T
- For every edge f of C , $\text{weight}(f) \leq \text{weight}(e)$

Proof:

- By contradiction
- If $\text{weight}(f) > \text{weight}(e)$ we can get a spanning tree of smaller weight by replacing e with f



Replacing f with e yields a better spanning tree



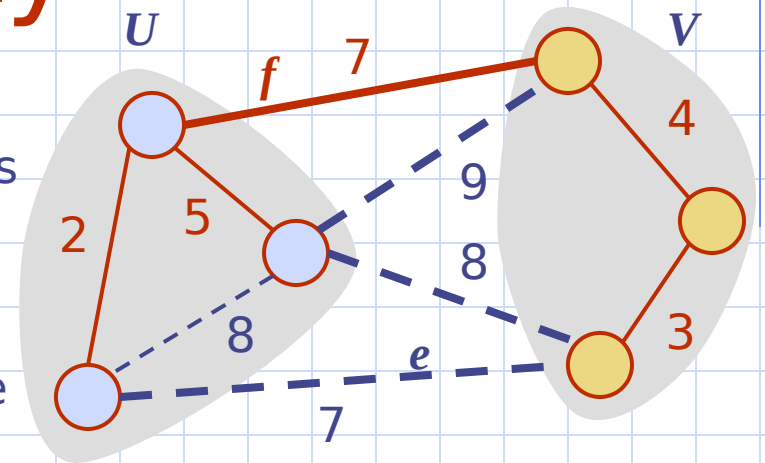
Partition Property

Partition Property:

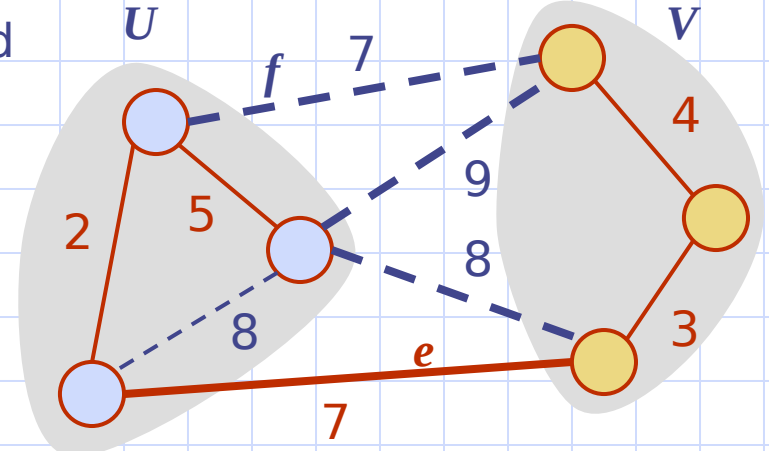
- Consider a partition of the vertices of G into subsets U and V
- Let e be an edge of minimum weight across the partition
- There is a minimum spanning tree of G containing edge e

Proof:

- Let T be an MST of G
- If T does not contain e , consider the cycle C formed by e with T and let f be an edge of C across the partition
- By the cycle property,
 $\text{weight}(f) \leq \text{weight}(e)$
- Thus, $\text{weight}(f) = \text{weight}(e)$
- We obtain another MST by replacing f with e



Replacing f with e yields another MST



Prim-Jarnik's Algorithm

- Similar to Dijkstra's algorithm
- We pick an arbitrary vertex s and we grow the MST as a cloud of vertices, starting from s
- We store with each vertex v label $d(v)$ representing the smallest weight of an edge connecting v to a vertex in the cloud
- At each step:
 - We add to the cloud the vertex u outside the cloud with the smallest distance label
 - We update the labels of the vertices adjacent to u

Prim-Jarnik Pseudo-code

Algorithm PrimJarnik(G):

Input: An undirected, weighted, connected graph G with n vertices and m edges

Output: A minimum spanning tree T for G

Pick any vertex s of G

$D[s] = 0$

for each vertex $v \neq s$ **do**

$D[v] = \infty$

Initialize $T = \emptyset$.

Initialize a priority queue Q with an entry $(D[v], (v, \text{None}))$ for each vertex v , where $D[v]$ is the key in the priority queue, and (v, None) is the associated value.

while Q is not empty **do**

$(u, e) = \text{value returned by } Q.\text{remove_min}()$

 Connect vertex u to T using edge e .

for each edge $e' = (u, v)$ such that v is in Q **do**

 {check if edge (u, v) better connects v to T }

if $w(u, v) < D[v]$ **then**

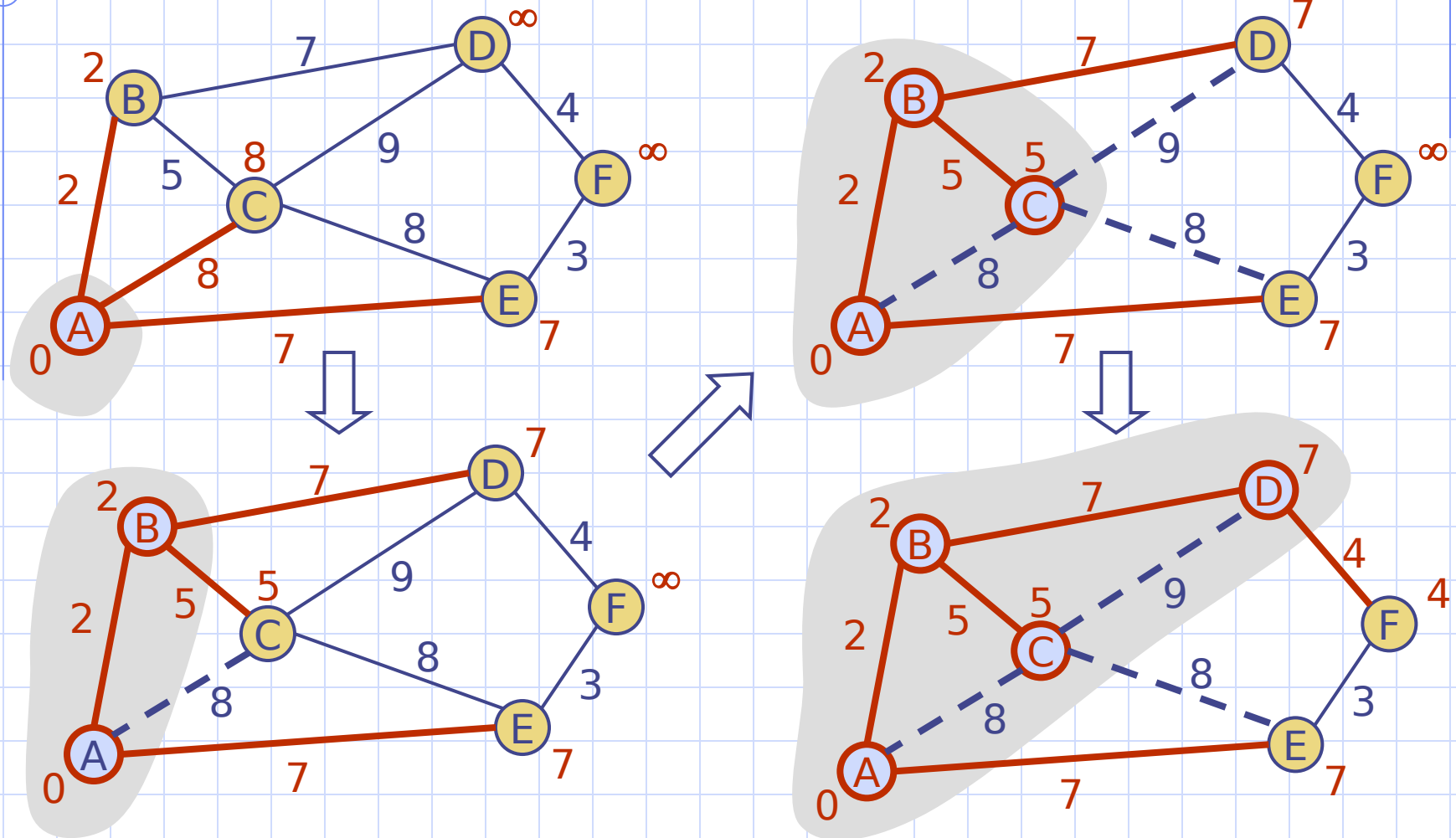
$D[v] = w(u, v)$

 Change the key of vertex v in Q to $D[v]$.

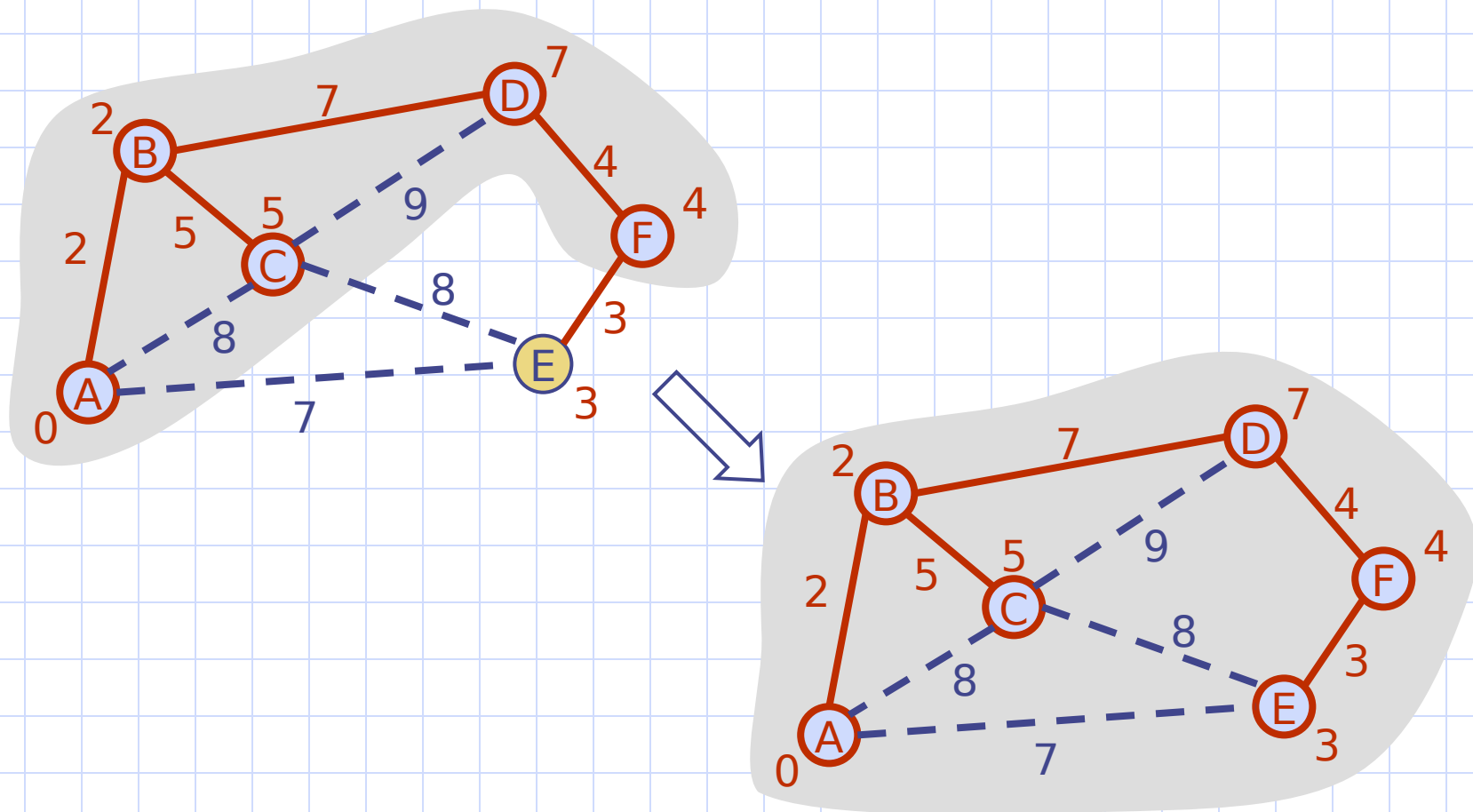
 Change the value of vertex v in Q to (v, e') .

return the tree T

Example



Example (contd.)



Analysis

- Graph operations
 - We cycle through the incident edges once for each vertex
- Label operations
 - We set/get the distance, parent and locator labels of vertex z $O(\deg(z))$ times
 - Setting/getting a label takes $O(1)$ time
- Priority queue operations
 - Each vertex is inserted once into and removed once from the priority queue, where each insertion or removal takes $O(\log n)$ time
 - The key of a vertex w in the priority queue is modified at most $\deg(w)$ times, where each key change takes $O(\log n)$ time
- Prim-Jarnik's algorithm runs in $O((n + m) \log n)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$
- The running time is $O(m \log n)$ since the graph is connected

Python Implementation

```
1 def MST_PrimJarnik(g):
2     """Compute a minimum spanning tree of weighted graph g.
3
4     Return a list of edges that comprise the MST (in arbitrary order).
5     """
6     d = { }                # d[v] is bound on distance to tree
7     tree = [ ]            # list of edges in spanning tree
8     pq = AdaptableHeapPriorityQueue( ) # d[v] maps to value (v, e=(u,v))
9     pqlocator = { }       # map from vertex to its pq locator
10
11     # for each vertex v of the graph, add an entry to the priority queue, with
12     # the source having distance 0 and all others having infinite distance
13     for v in g.vertices():
14         if len(d) == 0:    # this is the first node
15             d[v] = 0      # make it the root
16         else:
17             d[v] = float('inf') # positive infinity
18         pqlocator[v] = pq.add(d[v], (v, None))
19
20     while not pq.is_empty():
21         key, value = pq.remove_min()
22         u, edge = value    # unpack tuple from pq
23         del pqlocator[u]   # u is no longer in pq
24         if edge is not None:
25             tree.append(edge) # add edge to tree
26         for link in g.incident_edges(u):
27             v = link.opposite(u)
28             if v in pqlocator: # thus v not yet in tree
29                 # see if edge (u,v) better connects v to the growing tree
30                 wgt = link.element()
31                 if wgt < d[v]: # better edge to v?
32                     d[v] = wgt # update the distance
33                     pq.update(pqlocator[v], d[v], (v, link)) # update the pq entry
34     return tree
```

Kruskal's Approach

- Maintain a partition of the vertices into clusters
 - Initially, single-vertex clusters
 - Keep an MST for each cluster
 - Merge “closest” clusters and their MSTs
- A priority queue stores the edges outside clusters
 - Key: weight
 - Element: edge
- At the end of the algorithm
 - One cluster and one MST

Algorithm Kruskal(G):

Input: A simple connected weighted graph G with n vertices and m edges

Output: A minimum spanning tree T for G

for each vertex v in G do

Define an elementary cluster $C(v) = \{v\}$.

Initialize a priority queue Q to contain all edges in G , using the weights as keys.

$T = \emptyset$	$\{T \text{ will ultimately contain the edges of the MST}\}$
-----------------	--

while T has fewer than $n - 1$ edges **do**

(u, v) = value returned by $Q.remove_min()$

Let $C(u)$ be the cluster containing u , and let $C(v)$ be the cluster containing v .

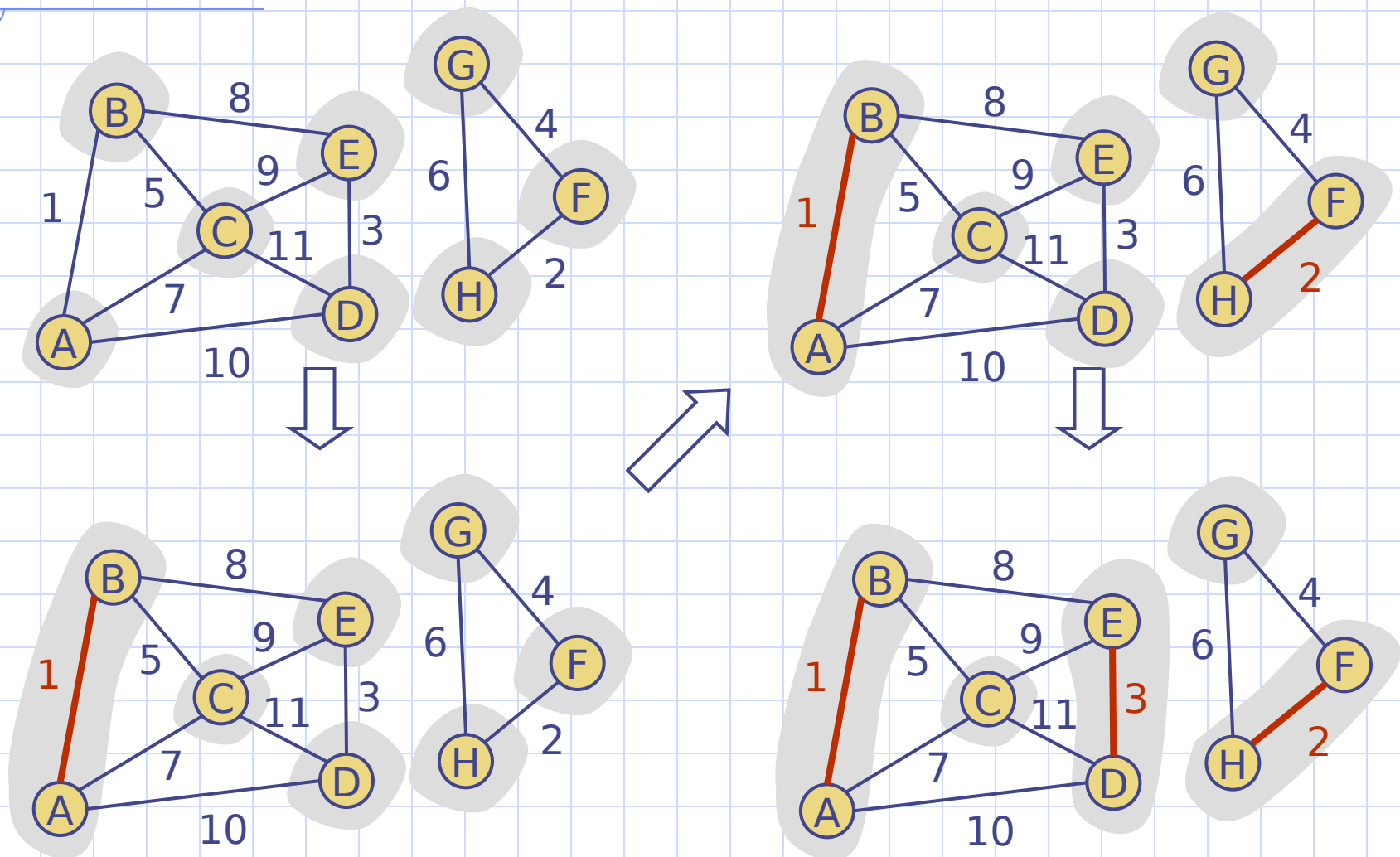
if $C(u) \neq C(v)$ then

Add edge (u, v) to T .

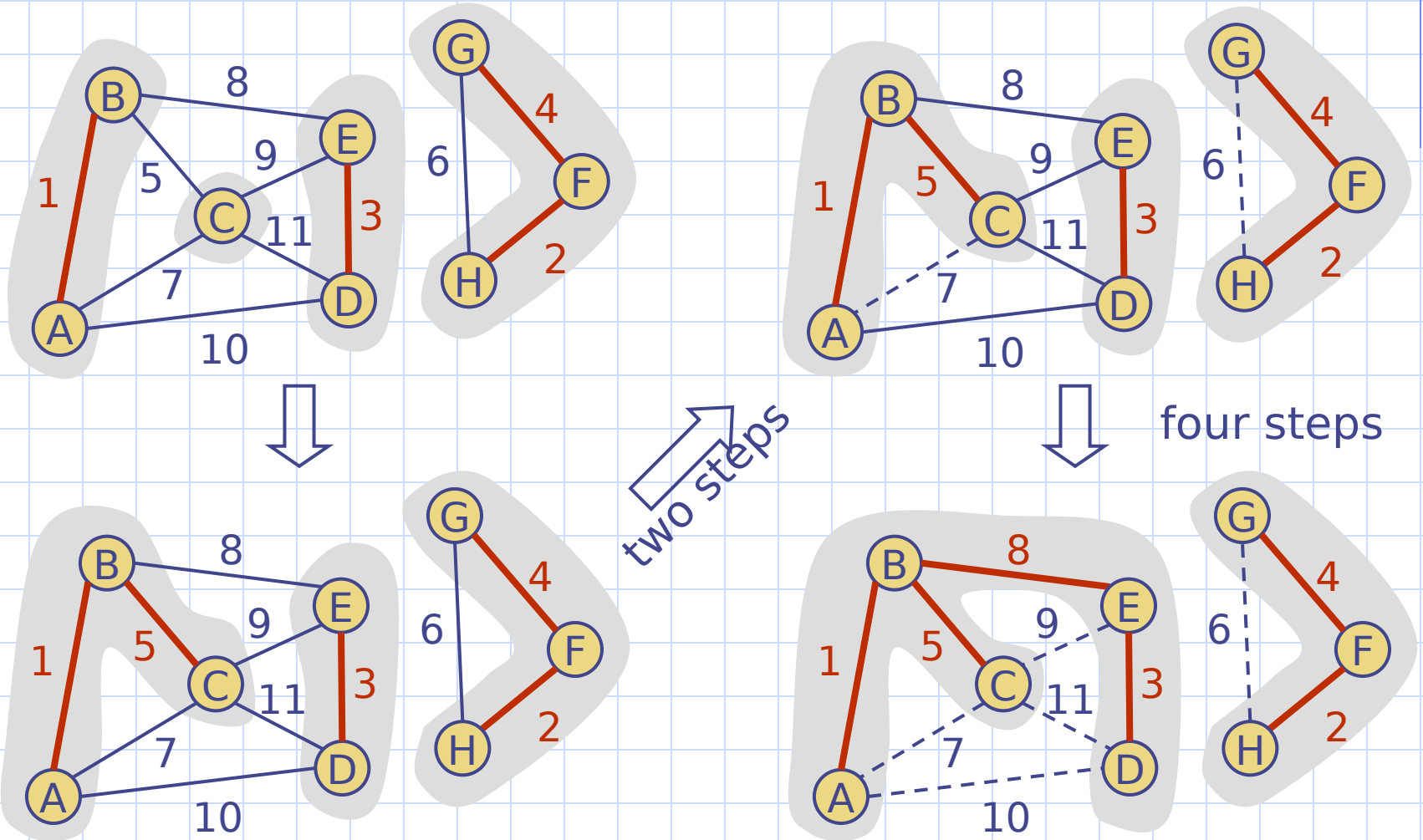
Merge $C(u)$ and $C(v)$ into one cluster.

return tree T

Example



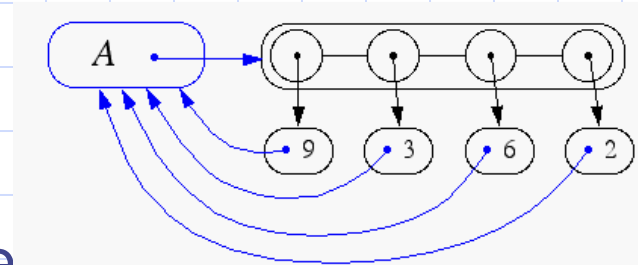
Example (contd.)



Data Structure for Kruskal's Algorithm

- ❑ The algorithm maintains a forest of trees
- ❑ A priority queue extracts the edges by increasing weight
- ❑ An edge is accepted if it connects distinct trees
- ❑ We need a data structure that maintains a **partition**, i.e., a collection of disjoint sets, with operations:
 - **makeSet**(u): create a set consisting of u
 - **find**(u): return the set storing u
 - **union**(A, B): replace sets A and B with their union

List-based Partition



- Each set is stored in a sequence
- Each element has a reference back to the set
 - operation **find**(u) takes $O(1)$ time, and returns the set of which u is a member.
 - in operation **union**(A, B), we move the elements of the smaller set to the sequence of the larger set and update their references
 - the time for operation **union**(A, B) is $\min(|A|, |B|)$
- Whenever an element is processed, it goes into a set of size at least double, hence each element is processed at most $\log n$ times

Partition-Based Implementation

- Partition-based version of Kruskal's Algorithm
 - Cluster merges as unions
 - Cluster locations as finds
- Running time $O((n + m) \log n)$
 - Priority Queue operations: $O(m \log n)$
 - Union-Find operations: $O(n \log n)$

Python Implementation

```
1 def MST_Kruskal(g):
2     """Compute a minimum spanning tree of a graph using Kruskal's algorithm.
3
4     Return a list of edges that comprise the MST.
5
6     The elements of the graph's edges are assumed to be weights.
7     """
8     tree = [ ]                # list of edges in spanning tree
9     pq = HeapPriorityQueue( ) # entries are edges in G, with weights as key
10    forest = Partition( )     # keeps track of forest clusters
11    position = { }           # map each node to its Partition entry
12
13    for v in g.vertices():
14        position[v] = forest.make_group(v)
15
16    for e in g.edges():
17        pq.add(e.element(), e) # edge's element is assumed to be its weight
18
19    size = g.vertex_count()
20    while len(tree) != size - 1 and not pq.is_empty():
21        # tree not spanning and unprocessed edges remain
22        weight, edge = pq.remove_min()
23        u, v = edge.endpoints()
24        a = forest.find(position[u])
25        b = forest.find(position[v])
26        if a != b:
27            tree.append(edge)
28            forest.union(a, b)
29
30    return tree
```